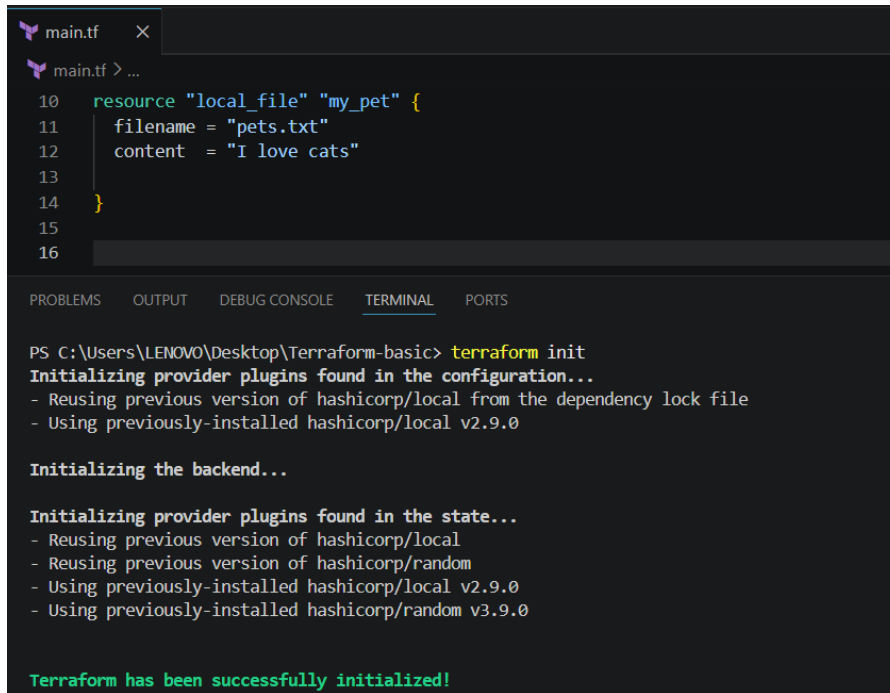


Terraform-04

1). Watch Terraform-04 video.

➤ Terraform Version Constraints



```
main.tf x
main.tf > ...
10 resource "local_file" "my_pet" {
11     filename = "pets.txt"
12     content  = "I love cats"
13 }
14
15
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

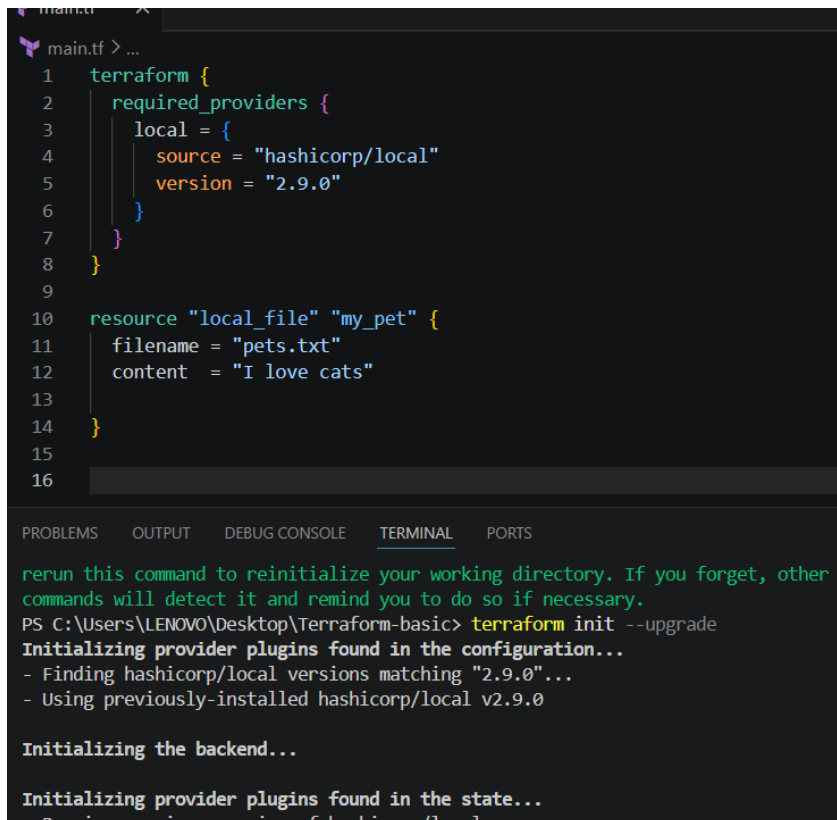
```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/local from the dependency lock file
- Using previously-installed hashicorp/local v2.9.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/local
- Reusing previous version of hashicorp/random
- Using previously-installed hashicorp/local v2.9.0
- Using previously-installed hashicorp/random v3.9.0

Terraform has been successfully initialized!
```

➤ Provider Version Constraints



```
main.tf x
main.tf > ...
1 terraform {
2     required_providers {
3         local = {
4             source = "hashicorp/local"
5             version = "2.9.0"
6         }
7     }
8 }
9
10 resource "local_file" "my_pet" {
11     filename = "pets.txt"
12     content  = "I love cats"
13 }
14
15
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init --upgrade
Initializing provider plugins found in the configuration...
- Finding hashicorp/local versions matching "2.9.0"...
- Using previously-installed hashicorp/local v2.9.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/local
```

```
main.tf x
main.tf > terraform
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "!=3.9.0"
6     }
7   }
8 }
9 resource "local_file" "my_pet" {
10  filename = "pets.txt"
11  content = "I love cats"
12 }
13 }
14 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/local from the dependency lock file
- Using previously-installed hashicorp/local v2.9.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/random
- Reusing previous version of hashicorp/local
- Using previously-installed hashicorp/random v3.9.0
- Using previously-installed hashicorp/local v2.9.0

Terraform has been successfully initialized!
```

➤ Data sources:

```
main.tf x dogs.txt pets.txt
main.tf > data "local_file" "dog" > abc filename
1 resource "local_file" "my-pet" {
2   filename = "pets.txt"
3   content = data.local_file.dog.content
4 }
5 data "local_file" "dog" {
6   filename = "dogs.txt"
7 }
```

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
data.local_file.dog: Reading...
data.local_file.dog: Read complete after 0s [id=da39a3ee5e6b4b0d3255bfef95601890afd80709]
local_file.my-pet[1]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[2]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
```

```
Plan: 1 to add, 0 to change, 3 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my-pet[2]: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Destruction complete after 0s
local_file.my-pet[2]: Destruction complete after 0s
local_file.my-pet: Destruction complete after 0s
local_file.my-pet: Creating...
local_file.my-pet: Creation complete after 0s [id=da39a3ee5e6b4b0d3255bfef95601890afd80709]

Apply complete! Resources: 1 added, 0 changed, 3 destroyed.
```

-

2). Execute the script shown in the video.

➤ Meta-Arguments:

```
main.tf X
main.tf > ...
1 resource "local_file" "my-pet" {
2   filename = "pets.txt"
3   content = "I love cats!"
4   count = 3
5 }
6
7

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my_pet: Refreshing state... [id=ba7d8973c074119188b5c1da0537e31a99b829cf]
random_pet.petname: Refreshing state... [id=MR.ferret]
```

```
Enter a value: yes

random_pet.petname: Destroying... [id=MR.ferret]
random_pet.petname: Destruction complete after 0s
local_file.my-pet[2]: Creating...
local_file.my-pet[0]: Creating...
local_file.my-pet[1]: Creating...
local_file.my-pet[2]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[0]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]

Apply complete! Resources: 3 added, 0 changed, 1 destroyed.
```

- Example of count: Argument

```
main.tf X variables.tf X
main.tf > ...
1 resource "local_file" "my-pet" {
2   filename = var.filename[count.index]
3   content = "I love cats!"
4   count = 3
5 }
6
7

variables.tf X
variables.tf > variable "filename" > {} default
1 variable "filename" {
2   default = [
3     "pets.txt",
4     "cats.txt",
5     "dogs.txt"
6   ]
7 }
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my-pet[0]: Refreshing state... [id=da39a3ee5e6b4b0d3255bfef95601890afd80709]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
+ create
-/+ destroy and then create replacement
```

Plan: 3 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
local_file.my-pet[0]: Destroying... [id=da39a3ee5e6b4b0d3255bfef95601890afd80709]
local_file.my-pet[0]: Destruction complete after 0s
local_file.my-pet[1]: Creating...
local_file.my-pet[2]: Creating...
local_file.my-pet[2]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[0]: Creating...
local_file.my-pet[0]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
```

Apply complete! Resources: 3 added, 0 changed, 1 destroyed.

- Now my count 3 and variables are 2 so it throws an error

```

main.tf
1 resource "local_file" "my-pet" {
2   filename = var.filename[count.index]
3   content = "I love cats!"
4   count = 3
5 }
6
7

variables.tf
1 variable "filename" {
2   default = [
3     "cats.txt",
4     "dogs.txt"
5   ]
6 }
7

# local_file.my-pet[1] must be replaced
-/+ resource "local_file" "my-pet" {
  ~ content_base64sha256 = "CqzQYr7q3D2Opjynmq/2H8Cy/3rdow20tw7dbRtqse4=" -> (known after apply)
  ~ content_base64sha512 = "C7VOW1Z8s4ghh//NuqTx8eycfZ2GK90k0oq2fptjj393DlFNE1Ab4BTlDdvHPVUK9bMsswoilV4/LK56+Em5tw==" -> (known after apply)
  ~ content_md5 = "9330b7e900600e5767c510f88a054a53" -> (known after apply)
  ~ content_sha1 = "c4956e2d4fae5b8edc05f4140566ad7a77210aa8" -> (known after apply)
  ~ content_sha256 = "0aacd062beeadc3d90a63ca79aaff61fc0b2ff7adda30db4b70eddd6d1b6ab1ee" -> (known after apply)
  ~ content_sha512 = "0bb54e5b567cb3882187ffcdbaa4f1fec9c7d967a2bdd243b4ab67e91e38f7f770e514d13501be01b4875dbc73ef50af5b32cb16a222d5e" -> (known after apply)
  ~ filename = "cats.txt" -> "dogs.txt" # forces replacement
  ~ id = "c4956e2d4fae5b8edc05f4140566ad7a77210aa8" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 2 to add, 0 to change, 2 to destroy.

Error: Invalid index

on main.tf line 2, in resource "local_file" "my-pet":
2: filename = var.filename[count.index]

count.index is 2
var.filename is tuple with 2 elements

```

SO, I used count length where to takes default how much length it has then it succeeds

```

main.tf
1 resource "local_file" "my-pet"
2   resource "local_file" "my-pet" {
3     filename = var.filename[count.index]
4     content = "I love cats!"
5     count = length(var.filename)
6   }
7

variables.tf
1 variable "filename" {
2   default = [
3     "cats.txt",
4     "dogs.txt"
5   ]
6 }
7

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my-pet[2]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[0]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy
-/+ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my-pet[0] must be replaced
-/+ resource "local_file" "my-pet" {

```

```

Enter a value: yes
local_file.my-pet[2]: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[0]: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Destroying... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[1]: Destruction complete after 0s
local_file.my-pet[0]: Destruction complete after 0s
local_file.my-pet[2]: Destruction complete after 0s
local_file.my-pet[1]: Creating...
local_file.my-pet[0]: Creating...
local_file.my-pet[1]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet[0]: Creation complete after 0s [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]

Apply complete! Resources: 2 added, 0 changed, 3 destroyed.

```

-

Using for-each also we can do and also type (string) in variable

```
main.tf x variables.tf ... variables.tf x
main.tf > resource "local_file" "my-pet" > filename
1 resource "local_file" "my-pet" {
2   filename = each.value
3   content = "I love cats!"
4   for_each = toset(var.filename)
5
6 }
7
8

variables.tf > ...
1 variable "filename" {
2   type = set(string)
3   default = [
4     "pets.txt",
5     "dogs.txt"
6   ]
7 }
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my-pet["dogs.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
local_file.my-pet["cats.txt"]: Refreshing state... [id=c4956e2d4fae5b8edc05f4140566ad7a77210aa8]
```

- AWS resources

```
main.tf x
main.tf > ...
1 resource "aws_iam_user" "Admin-user" {
2   name = "Arun"
3   tags = {
4     "description" = "Technical Team Lead"
5   }
6 }
7

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
```

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

- Deleted old Access and create new one

≡ IAM > Security credentials > Create access key

Step 1

Alternatives to root user access keys

Step 2

Retrieve access key

Retrieve access key [Info](#)

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key

Access key	Secret access key
AKIA5QKV7AB3S45C4ITG	***** Show

-
- After creating new access configure it and apply it

```
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\LENOVO\Desktop\Terraform-basic> aws configure
AWS Access Key ID [*****CV7X]: AKIA5QKV7AB3S45C4ITG
AWS Secret Access Key [*****9eC5]: 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
Default region name [us-east-1]:
Default output format [json]:
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
```

```
# aws_iam_user.Admin-user will be created
+ resource "aws_iam_user" "Admin-user" {
+   arn                = (known after apply)
+   force_destroy      = false
+   id                 = (known after apply)
+   name                = "Arun"
+   path                = "/"
+   tags                = {
+     "description" = "Technical Team Lead"
+   }
+   tags_all            = {
+     "description" = "Technical Team Lead"
+   }
+   unique_id           = (known after apply)
+ }
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

Created Arun id

```
aws_iam_user.Admin-user: Creating...
aws_iam_user.Admin-user: Creation complete after 2s [id=Arun]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```

- In IAM Users new User created

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

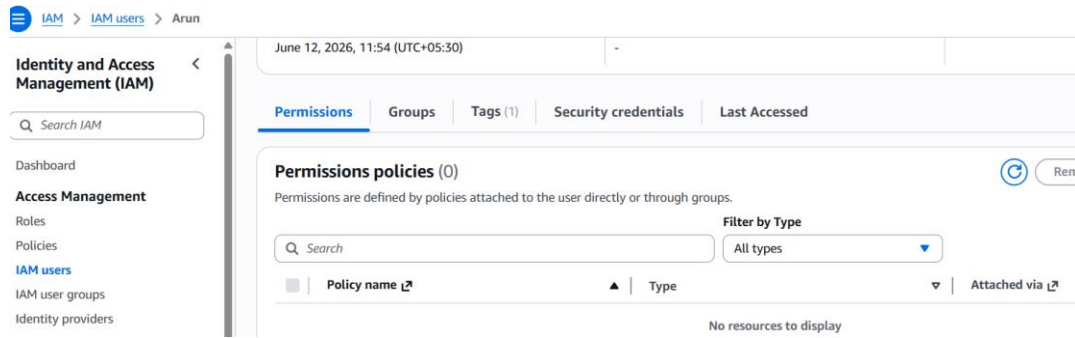
IAM users (2) [Info](#)

An IAM user is an identity with long-term credentials that is used to interact with AWS in a

Search

☐	User name	Path	Group:	La
☐	AdminUser	/	0	
☐	Arun	/	0	-

-
- Now do AWS resource to attach the policy for user (currently no policy has been attached)



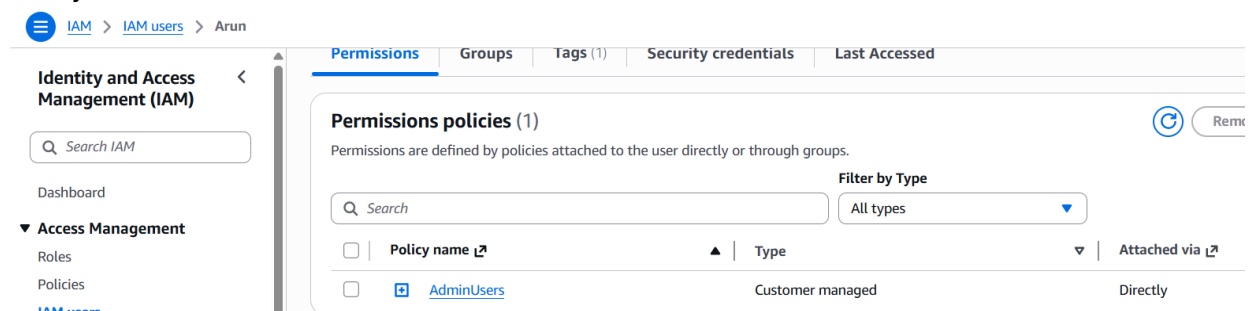
- Write a policy script and mention user name and policy to it apply and check in user

```
main.tf > resource "aws_iam_user_policy_attachment" "Arun-admin-access"
1 resource "aws_iam_user" "Admin-user" {
2   name = "Arun"
3   tags = {
4     "description" = "Technical Team Lead"
5   }
6 }
7 > resource "aws_iam_policy" "adminuser" { ...
10 {
11   "Version": "2012-10-17",
12   "Statement": [
13     {
14       "Sid": "1234567890",
15       "Effect": "Allow",
16       "Action": "*",
17       "Resource": "*"
18     }
19   ]
20 }
21 EOF
22 }
23
24 resource "aws_iam_user_policy_attachment" "Arun-admin-access" {
25   user = aws_iam_user.Admin-user.name
26   policy_arn = aws_iam_policy.adminuser.arn
27 }
```

- Terraform plan and apply

```
Enter a value: yes
aws_iam_policy.adminuser: Creating...
aws_iam_user.Admin-user: Creating...
aws_iam_user.Admin-user: Creation complete after 3s [id=Arun]
aws_iam_policy.adminuser: Creation complete after 4s [id=arn:aws:iam::928430096503:policy/AdminUsers]
aws_iam_user_policy_attachment.Arun-admin-access: Creating...
aws_iam_user_policy_attachment.Arun-admin-access: Creation complete after 1s [id=Arun/arn:aws:iam::928430096503:policy/AdminUsers]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

- Policy has been created



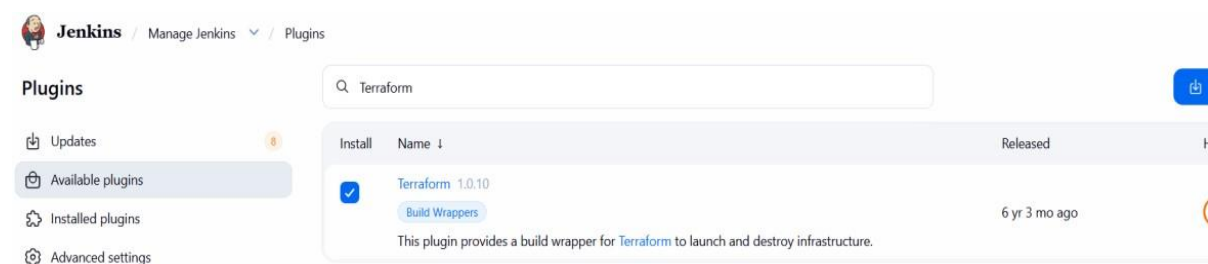
3). Integrate Terraform in Jenkins using the Terraform plugin.

- Open Jenkins server and check running or not and version also

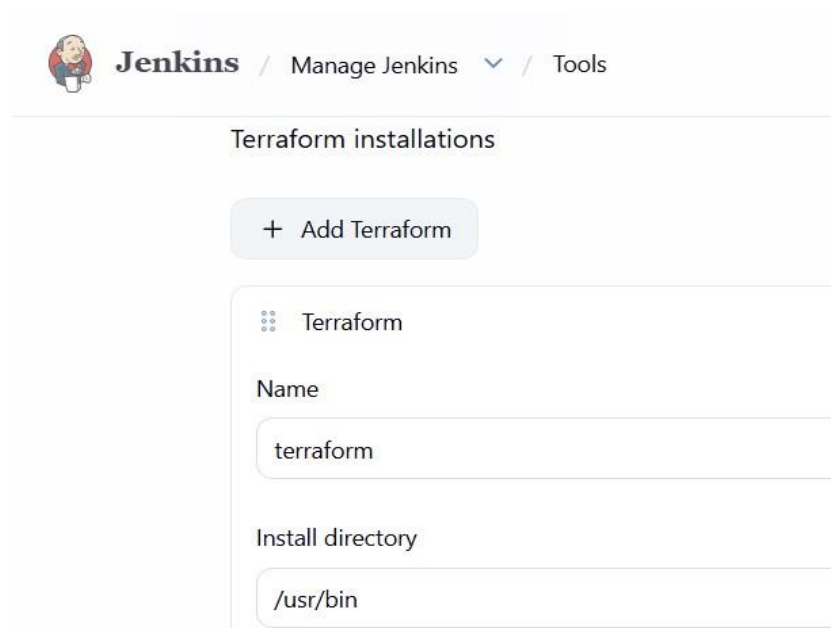
```
root@ip-172-31-22-87:~# systemctl start jenkins
root@ip-172-31-22-87:~# systemctl status jenkins
• jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-09 17:22:57 UTC; 37s ago
     Main PID: 519 (java)
       Tasks: 45 (limit: 1013)
      Memory: 389.3M (peak: 390.6M)
         CPU: 22.347s
        CGroup: /system.slice/jenkins.service
                └─519 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.wa
```

```
root@ip-172-31-22-87:~# terraform -v
Terraform v1.15.5
on linux_amd64
root@ip-172-31-22-87:~#
```

- Open Jenkins in browser and install Terraform plugin



- In Jenkins manage select Tools and add Terraform installations details



- Now select new job item and select pipeline

New Item

Enter an item name

Sample Terraform

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple

- Now in Jenkins server go to workspace of new created job and there create main.tf

```
root@ip-172-31-22-87:/var/lib/jenkins/workspace# cd ..
root@ip-172-31-22-87:/var/lib/jenkins# cd workspace/Sample\ Terraform
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# vi main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# ls
main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#
```

- Now in main.tf file write script and save it

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content  = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
```

- Now Pipeline and write a declarative script for the new created job

Configure

General

Triggers

Pipeline

Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Terraform Version') {
7       steps {
8         sh 'terraform -v'
9       }
10    }
11  }
```

- Build Success

```

15 / Sample Terraform / #1
}
  [32m+0m0m content_base64sha256 = (known after apply)
  [32m+0m0m content_base64sha512 = (known after apply)
  [32m+0m0m content_md5 = (known after apply)
  [32m+0m0m content_sha1 = (known after apply)
  [32m+0m0m content_sha256 = (known after apply)
  [32m+0m0m content_sha512 = (known after apply)
  [32m+0m0m directory_permission = "0777"
  [32m+0m0m file_permission = "0777"
  [32m+0m0m filename = "terraform-from-jenkins.txt"
  [32m+0m0m id = (known after apply)
}

[1mPlan:0m 0m1 to add, 0 to change, 0 to destroy.
[0m[1mlocal_file.demo: Creating...[0m[0m
[0m[1mlocal_file.demo: Creation complete after 0s [id=221065dac85086259e58924cd5d80a552d879e04]
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

- Now check in server successful

```

root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# terraform apply -auto-approve
local_file.demo: Refreshing state... [id=221065dac85086259e58924cd5d80a552d879e04]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#

```

- To check open Vi terraform-from-jenkins.txt

```

-rwxr-xr-x 1 jenkins jenkins 67 Jun 10 09:07 terraform-from-jenkins.txt
-rw-r--r-- 1 jenkins jenkins 1694 Jun 10 09:09 terraform.tfstate
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# vi terraform-from-jenkins.txt
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#

```

Successfully from Jenkins using Terraform plugin

```

Terraform executed successfully from Jenkins using Terraform Plugin
~
~
~

```

4). Create a CI/CD pipeline for a

Nodejs Application: <https://github.com/betawins/Trading-UI.git>

- Nodejs Installed and verify it

```

[root@ip-172-31-27-142 ~]# sudo dnf install nodejs -y
Last metadata expiration check: 0:07:10 ago on Fri Jun 12 06:48:03 2026.
Dependencies resolved.
=====
Package                                Architecture      Version
-----
Installing:
nodejs                                x86_64            1:18.20.8-1.amzn2023.0.2

```

```

[root@ip-172-31-27-142 ~]# curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo yum install -y nodejs
2026-06-12 06:55:44 - Cleaning up old repositories...
2026-06-12 06:55:44 - Old repositories removed
2026-06-12 06:55:44 - Supported architecture: x86_64
2026-06-12 06:55:44 - Added NTSolid repository for LTS version: 20.x

```

```
root@ip-172-31-22-87:~# node -v
v18.19.1
root@ip-172-31-22-87:~# npm -v
9.2.0
root@ip-172-31-22-87:~#
```

- Now open Jenkins Browser and create new pipeline

 **Jenkins** / All ▾ / New Item

New Item

Enter an item name

Trading-UI

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

- Now write pipeline script and save it and build now

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   environment {
5     NODE_OPTIONS = "--openssl-legacy-provider"
6     CI = "false"
7   }
8
9   stages {
10
11     stage('Checkout') {
12       steps {
13         git 'https://github.com/betawins/Trading-UI.git'
14       }
15     }
16   }
17 }
```

- Success Build

```
@[32m[PM2] @[39mSpawning PM2 daemon with pm2_home=/var/lib/jenkins/.pm2
@[32m[PM2] @[39mPM2 Successfully daemonized
@[1m@[34mUse --update-env to update environment variables@[39m@[22m
@[31m[PM2][ERROR] @[39mProcess or Namespace Trading-UI not found
+ pm2 start npm --name Trading-UI -- start
@[32m[PM2] @[39mStarting /usr/bin/npm in fork_mode (1 instance)
@[32m[PM2] @[39mDone.
```

id	name	namespace	version	mode	pid	uptime	U	status	cpu	mem	user	watching
[1m@[36m0	[39m@[22m	Trading-UI	default	N/A	[7m@[1mfork	[22m@[27m		3416	0s	0		
[32m@[1monline	[22m@[39m	0%	13.4mb	[1mjenkins	[22m	[90mdisabled	[39m					

```
+ pm2 save
@[32m[PM2] @[39mSaving current process list...
```

•

```

[32m[PM2] [39mSaving current process list...
[32m[PM2] [39mSuccessfully saved in /var/lib/jenkins/.pm2/dump.pm2
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Trading-UI Build and Deployment Successful
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Pipeline Script

```
pipeline {
  agent any

  environment {
    NODE_OPTIONS = "--openssl-legacy-provider"
    CI = "false"
  }

  stages {

    stage('Checkout') {
      steps {
        git 'https://github.com/betawins/Trading-UI.git'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh "
        npm install --legacy-peer-deps
        "
      }
    }

    stage('Build Application') {
      steps {
        sh "
        export NODE_OPTIONS=--openssl-legacy-provider
        npm run build
        "
      }
    }
  }
```

•

```
    }  
  }  
  
  stage('Deploy Application'){  
    steps {  
      sh '''  
        pm2 restart Trading-UI || pm2 start npm --name Trading-UI -- start  
        pm2 save  
      '''  
    }  
  }  
}  
  
post {  
  success {  
    echo 'Trading-UI Build and Deployment Successful'  
  }  
  failure {  
    echo 'Trading-UI Build Failed'  
  }  
}  
}
```

5). Explain 10 Maven commands.

1. **mvn compile** – Compiles source code.
2. **mvn test** – Runs test cases.
3. **mvn package** – Creates JAR/WAR file.
4. **mvn install** – Installs artifact into local repository.
5. **mvn clean** – Deletes target directory.
6. **mvn validate** – Validates project structure.
7. **mvn verify** – Runs checks and tests.
8. **mvn deploy** – Deploys artifact to remote repository.
9. **mvn dependency: tree** – Shows dependency hierarchy.
10. **mvn site** – Generates project documentation.